ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №6»

Выполнил:

Губатова Ангелина Владиковна

студент: 54-15 группы

Специальность: 09.02.08 Интеллектуальные интегрированные системы

Руководители: *Борисов Егор Николаевич*
Дадаев Матвей Васильевич

Оценка

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	**Губатова Ангелина Владиковна**

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №6»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта:

Сценарий	Склад зерна
Датчик 1 (аналоговый)	NTC (температура, 0-50 °C)
Датчик 2	DHT22 (влажность воздуха)
Датчик 3	PIR (движение у входа)
Исполнительное устройство	зуммер
Светодиод-индикатор (цвет, ток)	красный, 20 мА
Период опроса T	2 с
Порог события	влажность > 70 %
Дата выдачи задания для выполнения курсового проекта	«13» октября 2026 г.
Срок защиты курсового проекта	«5» ноября 2026 г.

Содержание курсового проекта

Введение
1. Постановка задачи и структурная схема
2. Принципиальная схема в EasyEDA и схема в Wokwi
3. Расчётная часть
    3.1. Расчёт резистора светодиода
    3.2. Расчёт шага АЦП
    3.3. Расчёт объёма данных
4. Программа микроконтроллера
5. База данных
6. Обработка данных в Node-RED
7. Тестирование и проверка неисправности
8. Предложения по модернизации

Выводы

Список используемых сокращений

Список используемой литературы

ПРИЛОЖЕНИЯ

А. Структурная схема
Б. Принципиальная схема (EasyEDA)
В. Схема в Wokwi (скриншот)
Г. Листинг скетча
Д. Схема базы данных (ER-диаграмма и schema.sql)
Е. Flow Node-RED (flow.json)

Список рекомендованной литературы и источников

1. Чакон С., Штрауб Б. Pro Git. Электронная книга: https://git-scm.com/book/ru/v2
2. Документация Wokwi: https://docs.wokwi.com
3. Справочник языка Arduino: https://www.arduino.cc/reference/
4. Документация Node-RED: https://nodered.org/docs/
5. Документация SQLite: https://www.sqlite.org/docs.html

Руководитель курсового проекта _____ (_____)

Подпись Расшифровка подписи

Студент _____ (_____)

Подпись Расшифровка подписи

Оглавление

Введение 6

1. Постановка задачи 8

2. Структура и алгоритм работы системы 10

2.1. Структурная схема 10

2.2. Алгоритм функционирования 10

2.3. Протокол обмена данными 11

3. Разработка аппаратной части 12

3.1. Выбор компонентов и подключение 12

3.2. Расчёт резисторов 12

3.3. Принципиальная схема в EasyEDA 13

3.4. Модель в Wokwi 14

4. Разработка прошивки 15

5. Проектирование базы данных 16

5.1. Модель данных 16

5.2. Описание таблиц 16

5.3. Контрольные запросы 17

6. Загрузка и контроль данных в Node-RED 18

6.1. Состав потока 18

6.2. Функция разбора журнала 18

6.3. Запуск 19

7. Тестирование и результаты 20

7.1. Штатный сценарий 20

7.2. Сценарий неисправности датчика 21

7.3. Повторная загрузка 22

7.4. Сводка результатов 22
8. Расчёт объёма данных 23
9. Предложения по модернизации 24
10. Управление версиями Git 25
Выводы 26
Список используемых сокращений 27
Список используемой литературы 28
ПРИЛОЖЕНИЕ А 29
ПРИЛОЖЕНИЕ Б 30
ПРИЛОЖЕНИЕ В 31
Приложение Г Модель в Wokwi 32
Приложение Д Листинг программы Arduino (variant_6.ino) 33
Приложение Е Листинг функции разбора журнала Node-RED 35
Приложение Ж Скрипты базы данных 37
Приложение И Журнал Serial и загрузочный 38
Приложение К 40

Введение

Интеллектуальные интегрированные системы (ИИС) строятся на основе датчиков, исполнительных устройств и программируемых контроллеров, объединённых каналами передачи данных и средствами хранения информации. Типовая задача сопровождения таких систем состоит в том, чтобы получать показания датчиков, сохранять их, находить отклонения от нормы и выдавать результат для анализа.

Прежде чем собирать устройство из реальных компонентов, его проектируют и проверяют на модели. Принципиальную схему разрабатывают в системе автоматизированного проектирования EasyEDA, работу программы микроконтроллера проверяют в симуляторе Wokwi, результаты измерений сохраняют в базе данных SQLite, а за загрузку и контроль данных отвечает среда потоковой разработки Node-RED. Все файлы проекта ведутся в системе контроля версий Git. Такой набор программ не требует установки оборудования и позволяет воспроизвести проект на любом компьютере.

Объектом курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Склад зерна». Предмет проекта - схемотехника, программа микроконтроллера, структура базы данных и средства загрузки данных, обеспечивающие контроль температуры, влажности воздуха и наличия движения на складе.

Цель проекта - разработать и документировать работающую виртуальную систему, в которой показания трёх датчиков через последовательный порт попадают в журнал, из журнала автоматически загружаются в базу данных и проверяются на выход за допустимые пределы.

Для достижения цели необходимо решить следующие задачи:

- разработать структурную схему и алгоритм работы системы

-выбрать компоненты, рассчитать токоограничивающие резисторы и выполнить принципиальную схему в EasyEDA по требованиям ЕСКД

- собрать модель в Wokwi и написать программу для Arduino Uno

- спроектировать базу данных SQLite и набор контрольных запросов

- настроить поток Node-RED, который разбирает журнал, загружает данные в базу и фиксирует неисправности

- провести тестирование на штатном и аварийном сценариях

- оформить проект в репозитории Git и подготовить пояснительную записку

1. Постановка задачи

В соответствии с заданием варианта №6 я разрабатываю систему контроля условий хранения на складе зерна. Первый датчик измеряет температуру воздуха, второй определяет влажность воздуха, третий фиксирует движение у входа. Исполнительным устройством служит зуммер, сигнализирующий о превышении влажности. Входные данные варианта приведены в таблице 1.

Таблица 1 - Вводные данные

Сценарий	Склад зерна
Датчик 1 (аналоговый)	NTC (температура, 0-50 °C)
Датчик 2	DHT22 (влажность воздуха)
Датчик 3	PIR (движение у входа)
Исполнительное устройство	зуммер
Светодиод-индикатор (цвет, ток)	красный, 20 мА
Период опроса T	2 с
Порог события	влажность > 70 %

Система должна выполнять следующие функции:

1) каждые T= 2 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA»;

2) при повышении влажности выше 70 % включать зуммер и красный светодиод и передавать строку «EVENT» с типом THRESHOLD; при возврате влажности в норму выключать их и передавать EVENT с типом NORMAL;

3) сохранять строки последовательного порта в текстовый журнал data/log.txt;

4) автоматически загружать журнал в базу SQLite (таблицы readings и events);

5) при значении датчика вне допустимого диапазона записывать в таблицу событий неисправность FAULT;

6) выдавать три контрольных запроса: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий.

Результатами работы являются: принципиальная схема в EasyEDA, модель в Wokwi, программа Arduino, база данных, поток Node-RED, репозиторий Git и настоящая пояснительная записка с приложениями.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных (рисунок 1). Датчики подключены к входам платы; плата управляет зуммером и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.

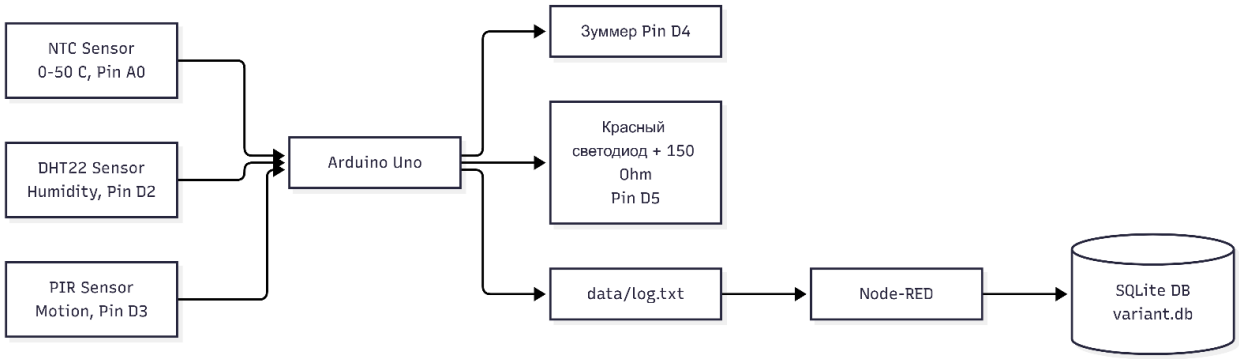


Рисунок 1 - Структурная схема системы

Структурная схема в увеличенном виде приведена в приложении А.

2.2. Алгоритм функционирования

Программа микроконтроллера выполняется циклически. После инициализации порта и выводов зуммер и светодиод выключены, номер измерения n= 0, признак превышения порога alarm= ложь. В каждом цикле увеличивается номер измерения, считываются три датчика, и в порт передаётся строка DATA.

Затем проверяется условие события s2 > 70 %. Событие формируется только в момент смены состояния: если условие выполнено, а признак alarm сброшен, включаются зуммер и светодиод и передаётся EVENT с типом THRESHOLD; если условие не выполнено, а alarm установлен, исполнительные устройства выключаются и передаётся EVENT с типом NORMAL. Такой подход исключает повторную отправку одного и того же события в каждом цикле. В конце цикла выдерживается пауза T= 2 с.

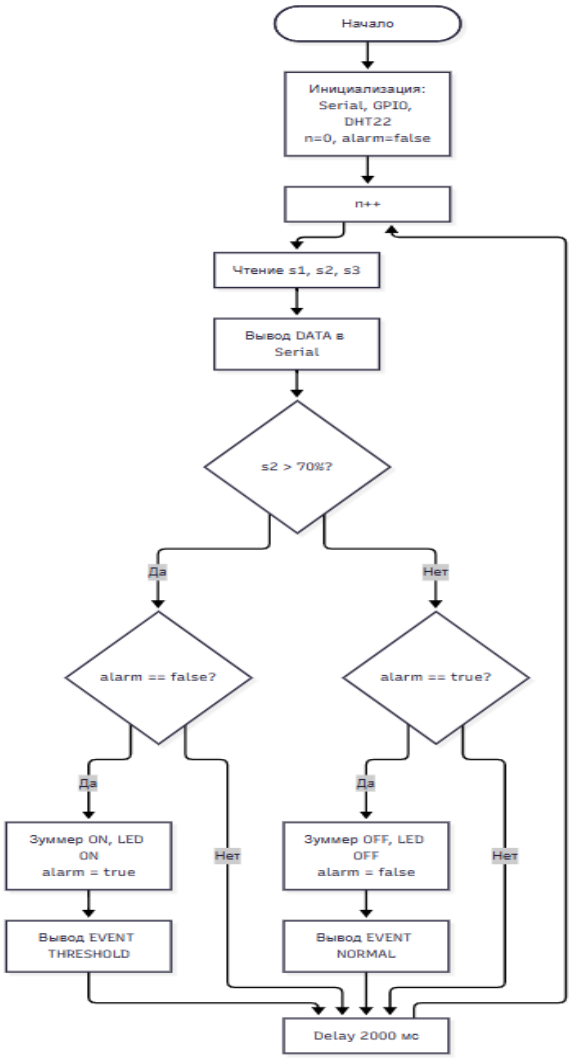


Рисунок 2 - Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках, которые одинаково читаются человеком и разбираются программой. Скорость порта – 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 - Формат строк последовательного порта

Тип	Формат	Пример
DATA	DATA n=<номер> s1=<температура> s2=<влажность> s3=<движение>	DATA n=4 s1=24.50 s2=61.0 s3=1
EVENT	EVENT n=<номер> type=THRESHOLD | NORMAL	EVENT n=4 type=THRESHOLD

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты, которые есть в библиотеках EasyEDA и Wokwi: плата Arduino Uno на микроконтроллере ATmega328P; термистор NTC, имитирующий датчик температуры; датчик влажности и температуры DHT22; пассивный инфракрасный датчик движения HC-SR501 (PIR); пассивный зуммер; красный светодиод индикатора. Подключение к выводам платы приведено в таблице 3.

Таблица 3 - Подключение компонентов к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Температура (s1)	NTC термистор + резистор делителя	A0	аналоговый вход
Влажность (s2)	DHT22 Data	D4	цифровой вход/выход
Движение (s3)	PIR Sensor Out	D2	цифровой вход
Зуммер	BZ1	D7	выход
Индикатор	HL1, красный светодиод через R1	D8	выход
Питание	Все модули	5 V, GND	–

NTC термистор включён в схеме делителя напряжения с постоянным резистором 10 кОм между 5 В и общим проводом, точка соединения подключена к A0. Десятиразрядный АЦП преобразует напряжение 0–5 В в код 0–1023. Для симуляции в Wokwi используется потенциометр или блок NTC с заданным сопротивлением.

Датчик DHT22 подключен к цифровому выводу D4. Датчик PIR подключен к выводу D2. Зуммер и светодиод управляются цифровыми выходами D7 и D8

3.2. Расчёт резисторов

Резистор R1 ограничивает ток красного светодиода HL1, подключённого к выводу D8 (выходное напряжение 5 В). Прямое падение напряжения красного

светодиода принимаем равным Uпр= 2,0 В (значение из методических указаний), рабочий ток по заданию I= 20 мА:

$$R1 = \frac{U_{\text{пит}} - U_{\text{пр}}}{I} = \frac{5 - 2{,}0}{0{,}020} = 150 \text{ Ом}$$

Расчётное значение 150 Ом соответствует стандартному номиналу ряда E24. Фактический ток I = (5 – 2,0)/ 150 = 20 мА. Мощность, рассеиваемая резистором, $P = I^2 \cdot R = 0,02^2 \cdot 150 = 0{,}06$Вт, достаточно резистора 0,25 Вт. Ток 20 мА не превышает допустимых 40 мА для вывода ATmega328P.

Таблица 4- оценка потребляемого тока

Потребитель	**Ток, мА**	**Примечание**
Плата Arduino Uno (ATmega328P)	50	типовое
Датчик DHT22	1,5	по документации
Датчик PIR (HC-SR501)	0,065	< 65 мкА
Зуммер (BZ1)	30	при включении
Светодиод HL1 (красный) с резистором R1	20	расчётное значение
Термистор NTC (через делитель 10 кОм)	0,5	$I = 5$ В/10 кОм
Итого	**≈ 102**	< 500 (USB)

3.3. Принципиальная схема в EasyEDA

Схема электрическая принципиальная выполнена в EasyEDA на листе формата А4 с основной надписью по ГОСТ 2.104-2006 (форма 1). Условные графические обозначения соответствуют ГОСТ 2.721-74 и ГОСТ 2.730-73, позиционные обозначения – ГОСТ 2.710-81: U1 – плата Arduino, U2 – NTC, U3 – DHT22, U4 – PIR, LED1 – светодиод, R1 – резистор, BZ1 – зуммер. Схема приведена в приложении Б, перечень элементов – в приложении В.

Порядок построения схемы: сначала в схему вводятся компоненты из библиотеки и задаются позиционные обозначения; затем проводятся цепи питания (5 V, GND) и сигнальные цепи в соответствии с таблицей 3; каждая цепь получает сетевую метку; после этого выполняется проверка правил (Design Manager → ERC), и схема экспортируется в PNG для вставки в рамку.

3.4. Модель в Wokwi

После утверждения схемы выполнена модель в симуляторе Wokwi (приложение Г). В проект добавлены элементы wokwi-arduino-uno, wokwi-ntc-thermistor (или потенциометр для эмуляции), wokwi-dht22, wokwi-pir-motion-sensor, wokwi-buzzer, wokwi-led и резистор. Соединения повторяют схему EasyEDA. Температура задаётся изменением сопротивления NTC, влажность – настройками DHT22, движение – переключением PIR.

Wokwi работает в браузере и не может записывать файлы на диск компьютера. Поэтому строки, выводимые в Serial Monitor, копируются и сохраняются в файл data/log.txt вручную; дальнейшая загрузка в базу данных выполняется автоматически (раздел 6).

4. Разработка прошивки

Программа для Arduino Uno написана на языке C++ в среде Wokwi; полный текст приведён в приложении Д. Параметры варианта вынесены в константы в начале программы: номера выводов, период опроса T_MS= 2000 мс, диапазон датчика влажности S2_MIN= 0 и S2_MAX= 100, порог THRESHOLD= 70,0 %.

Для чтения данных с DHT22 используется библиотека DHT.h (в симуляторе Wokwi она поддерживается). Для NTC используется функция analogRead, преобразующая код АЦП в температуру по линейной аппроксимации для учебного диапазона 0-50°C.

Таблица 5 - Функции программы

Функция	**Назначение**
setup():	настройка порта (9600 бод) и режимов выводов, начальное выключение зуммера и светодиода.
loop():	основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T.
readS1():	чтение температуры с NTC.
readS2():	чтение влажности с DHT22.
readS3():	чтение состояния PIR
actuatorOn()/actuatorOff():	включение и выключение зуммера и светодиода.

Для передачи данных использованы функции Serial.print() и Serial.println(): температура выводится с двумя знаками после запятой, влажность – с одним, движение – целым числом. Такой формат совпадает с протоколом из таблицы 2 и разбирается в Node-RED без дополнительных преобразований.

5. Проектирование базы данных

5.1. Модель данных

Для хранения данных выбрана встраиваемая СУБД SQLite: база целиком хранится в одном файле db/variant.db, не требует сервера и работает под управлением консольной программы sqlite3. Модель состоит из двух таблиц (рисунок 3): readings – показания датчиков, events – события и неисправности.

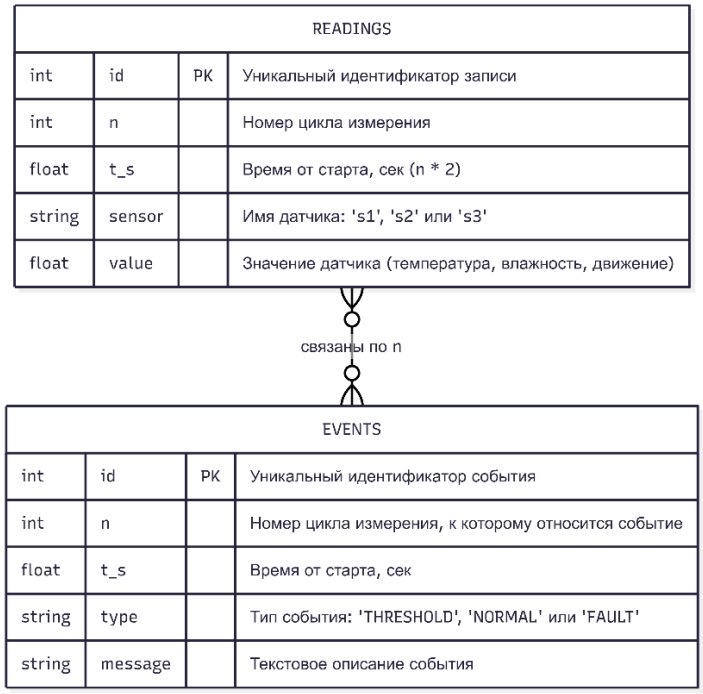


Рисунок 3 - Модель данных

Показания хранятся в «длинном» формате: один датчик в одном цикле – одна строка. Благодаря этому структура таблицы не зависит от числа датчиков, а запросы по конкретному датчику выполняются условием по полю sensor. Связь между таблицами логическая: по номеру измерения n.

5.2. Описание таблиц

Таблица 6 - Структура таблицы readings

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения из журнала
t_s	INTEGER, NOT NULL	время от начала измерений, с (t_s= n·T)
sensor	TEXT, NOT NULL	датчик: s1, s2 или s3
value	REAL, NOT NULL	значение датчика

Таблица 7 - Структура таблицы events

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения, при котором возникло событие
t_s	INTEGER, NOT NULL	время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	пояснение (для FAULT – какой датчик неисправен)

Скрипт создания таблиц db/schema.sql использует оператор CREATE TABLE IF NOT EXISTS, поэтому его можно выполнять повторно без ошибок. Текст скрипта приведён в приложении Ж.

5.3. Контрольные запросы

Для проверки данных составлены три запроса (файл db/queries.sql, приложение Ж).

1. Запрос выводит десять последних показаний и показывает, что загрузка дошла до конца журнала.
2. Запрос вычисляет минимум, среднее и максимум по каждому датчику с помощью агрегатных функций MIN, AVG, MAX и группировки GROUP BY sensor.
3. Запрос выводит все события по порядку их возникновения.

1. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Загрузка данных выполнена потоком Node-RED, показанным на рисунке 4. Поток следит за файлом data/log.txt и при каждом сохранении файла автоматически перезагружает базу данных. Дополнительно предусмотрена кнопка ручного запуска. Назначение узлов приведено в таблице 8.

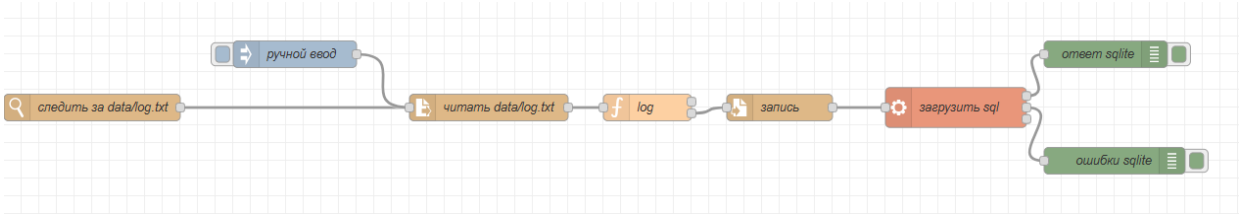


Рисунок 4 - Поток Node-Red

Таблица 8 - Узлы потока

Узел	Тип	Настройка и назначение
Следить за data/log.txt	watch	Формирует сообщение при каждом сохранении файла журнала
Загрузить лог вручную	inject	Ручной запуск потока кнопкой
Читать data/log.txt	file in	Читает весь текст журнала одной строкой
Разбор лога	function	Разбирает строки DATA и EVENT, проверяет диапазоны датчиков, формирует SQL-команды.
Записать db/load.sql	file	Записывает SQL-скрипт загрузки (подключается ко второму выходу функции)
Загрузить в базу	exec	Выполняет загрузку данных в базу
Ответ sqlite3	debug	Вывод штатного ответа программы sqlite3 в панель Debug
Ошибки sqlite3	debug	Вывод ошибок sqlite3 (второй и третий выходы узла exec) в панель Debug

6.2. Функция разбора журнала

Функция (приложение Е) получает текст журнала и делит его на строки. Каждая строка разбивается по пробелам на тип сообщения (DATA или EVENT) и пары «ключ=значение». Номер измерения n преобразуется в число, время вычисляется как $t_s = n \cdot 2$.

Для строки DATA для каждого из датчиков s1, s2, s3 формируется команда INSERT в таблицу readings. Значение проверяется по допустимому диапазону:

- s1 (температура): 0–50
- s2 (влажность): 0–100
- s3 (движение): 0–1

Если значение отсутствует или выходит за границы, добавляется команда INSERT в таблицу events с типом FAULT и текстом «датчик sN вне диапазона». Для строки EVENT формируется запись в events с типом, переданным контроллером.

Функция имеет два выхода. Второй выдаёт единый скрипт load.sql, который начинается с команд BEGIN и очистки таблиц (DELETE FROM readings, DELETE FROM events, сброс счётчиков sqlite_sequence) и заканчивается командой COMMIT. Очистка делает загрузку повторяемой: при повторной загрузке того же журнала записи не дублируются.

6.3. Запуск

Работа ведётся на компьютере без прав администратора, все программы портативные и лежат в папке %USERPROFILE%\tools. Командный файл startnodered.bat перед запуском переходит в папку проекта, благодаря чему относительные пути data/log.txt и db/variant.db указываются от корня проекта. Порядок запуска: выполнить start-nodered.bat, открыть редактор по адресу http://localhost:1880, импортировать поток из файла nodered/flow.json и нажать «Deploy». После сохранения файла data/log.txt загрузка происходит автоматически; результат проверяется командой: sqlite3 db/variant.db ".read db/queries.sql"

2. Тестирование и результаты

7.1. Штатный сценарий

Для проверки смоделирован процесс изменения условий на складе (12 циклов, 24 с). В циклах 1–3 влажность в норме (60%), температура стабильна. В цикле 4 влажность повышается до 75% – формируется событие THRESHOLD, включаются зуммер и светодиод. В цикле 8 влажность снижается до 65% – формируется событие NORMAL. Журнал Serial Monitor приведён в таблице 6. Для проверки смоделирован процесс изменения условий на складе (12 циклов, 24 с). В циклах 1–3 влажность в норме (60%), температура стабильна. В цикле 4 влажность повышается до 75% – формируется событие THRESHOLD, включаются зуммер и светодиод. В цикле 8 влажность снижается до 65% – формируется событие NORMAL. Журнал Serial Monitor приведён в таблице 6.

Таблица 9 - Журнал штатного сценария

n	t_s, c	s1, °C	s2, %	s3	Событие
1	2	25.56	91.50	0	THRESHOLD
2	4	25.56	91.50	0	–
3	6	25.56	91.50	0	–
4	8	25.56	91.50	0	–
5	10	23.51	91.50	0	–
6	12	23.51	91.50	0	–
7	14	23.51	91.50	0	–
8	16	23.51	57.50	0	NORMAL
9	18	23.51	57.50	0	–
10	20	23.51	57.50	0	–
11	22	23.51	57.50	0	–
12	24	23.51	57.50	0	–
13	26	23.51	57.50	0	–
1	2	25.56	91.50	0	THRESHOLD

После сохранения журнала Node-RED загрузил данные в базу. Счётчики записей: в таблице readings 36 строк (12 циклов по 3 датчика), в таблице events – 2 строки. Результаты контрольных запросов подтверждают корректность данных: влажность варьировалась от 60 до 75%, события THRESHOLD и NORMAL зафиксированы верно.

Таблица 10 - Запрос 1: десять последних показаний

id	n	t_s	sensor	value
39	13	26	s3	0
38	13	26	s2	57,50
37	13	26	s1	23,51
36	12	24	s3	0
35	12	24	s2	57,50
34	12	24	s1	23,51
33	11	22	s3	0
32	11	22	s2	57,50
31	11	22	s1	23,51
30	10	20	s3	0

Таблица 11 - Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	23,51	24,06	25,56
s2	57,50	76,58	91,50
s3	0	0	0

Среднее, минимум и максимум на датчике s3 равны нулю, так как движение не зафиксировано.

Таблица 12 - Запрос 3: все события

id	n	t_s	type	message
1	1	2	THRESHOLD	–
2	8	16	NORMAL	–

События совпали с моделируемой ситуацией: **THRESHOLD** в цикле 1 ($t_s = 2$ с), так как начальная влажность (91,50 %) уже превышала порог 70 %, и **NORMAL** в цикле 8 ($t_s = 16$ с), когда влажность снизилась до 57,50 %. Неисправностей FAULT нет

7.2. Сценарий неисправности датчика

Для проверки контроля диапазонов в журнал подана строка, в которой датчик влажности передаёт недопустимое значение 999 (диапазон 0–100 %): DATA n=1 s1=25.0 s2=120.0 s3=0

После загрузки в таблице readings появились 3 строки, а в таблице events – 1 строка: n= 1, t_s = 2, type = FAULT, message = «датчик s2 вне диапазона».

7.3. Повторная загрузка

Журнал штатного сценария сохранён повторно без изменений для проверки корректности работы скрипта загрузки load.sql. Счётчики записей остались прежними: 39 строк в таблице readings и 2 строки в таблице events. Это подтверждает, что команды очистки таблиц (DELETE FROM readings, DELETE FROM events) в начале SQL-скрипта предотвращают дублирование данных при повторном выполнении потока Node-RED.

7.4. Сводка результатов

Таблица 13 - Результаты проверок

№	Проверка	Ожидаемый результат	Получено	Итог
1	Загрузка штатного журнала	39 показаний, 2 события	39 и 2	пройден
2	Событие при влажности > 70 %	THRESHOLD при n= 1	n= 1, t_s= 2	пройден
3	Возврат влажности в норму	NORMAL при n= 8	n= 8, t_s= 16	пройден
4	Значение вне диапазона	FAULT, 3 показания	1 FAULT, 3 показания	пройден
5	Повторная загрузка	счётчики не растут	39 и 2	пройден
6	Работа BZ1 и HL1 в Wokwi	включены при влажности > 70 %	включались и выключались по событиям	пройден

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T= 2 с приведён в таблице 7.

Таблица 14 - Обьем данных

Показатель	Расчёт	Результат
Циклов в сутки	86 400 с / 2 с	43 200
Записей readings в сутки	43 200 · 3 датчика	129 600
Размер строки в базе (оценка)	id, n, t_s, sensor, value	≈ 40 Б
Объём базы за сутки	129 600 · 40 Б	≈ 5,2 МБ
Размер строки журнала	DATA n=43200 s1=50.00 s2=100.0 s3=0 + CRLF	≈ 36 Б
Объём журнала за сутки	43 200 · 36 Б	≈ 1,6 МБ
Объём базы за 30 суток	5,2 МБ · 30	≈ 156 МБ
Объём базы за год	5,2 МБ · 365	≈ 1,9 ГБ

Объём базы данных за год близок к 2 ГБ, что для SQLite не является проблемой — ограничения СУБД намного выше, а практический предел определяется свободным местом на диске. При длительной эксплуатации рекомендуется архивировать журнал и периодически удалять показания старше заданного срока.

9. Предложения по модернизации

На основе разработанной виртуальной модели можно предложить несколько направлений модернизации системы для её внедрения в реальные условия эксплуатации на складе зерна:

1. **Автоматизация климат-контроля.** В текущей версии при превышении порога влажности (> 70 %) система лишь сигнализирует об этом (включает зуммер и светодиод). Для реального склада необходимо добавить исполнительные устройства для активного воздействия на среду. Например, подключение через силовое реле промышленного вентилятора или сервопривода для автоматического открытия вентиляционных люков при наступлении события THRESHOLD. Это переведёт систему из режима пассивного мониторинга в режим активного управления микроклиматом.
2. **Визуализация данных и облачный мониторинг.** Интеграция потока Node-RED с облачными платформами. Это позволит построить интерактивные дашборды с графиками изменения температуры и влажности за выбранный период, а также настроить отправку push-уведомлений или SMS на смартфон ответственного сотрудника при возникновении критических событий (THRESHOLD) или неисправностей датчиков (FAULT).

10. Управление версиями Git

Проект ведётся в локальном репозитории Git. Структура папки проекта: firmware (программа Arduino), db (скрипты базы данных и запросы), nodered (экспорт потока), data (журнал), sim (модель Wokwi и скриншоты), docs (схема EasyEDA, пояснительная записка), файлы start-nodered.bat, env.bat и README.md. Файл базы db/variant.db и промежуточный db/load.sql создаются автоматически, поэтому в репозиторий не включаются (файл .gitignore).

Порядок работы:

git init

git config user.name "Gubatova Angelina"

git config user.email "angelinagubatova@gmail.com"

git add .

git commit -m "Структура проекта"

... (по одному коммиту на этап)

git tag v1.0

git log –oneline

Хеш	Дата	Сообщение коммита	Этап / Примечание
b9a25a6	29.10.2026	Initial commit	Создание репозитория, структура папок, README.md
c076db0	29.10.2026	добавлена структурная схема	docs/structure.png
6614e2f	02.11.2026	Добавлена схема в Wokwi	sim/diagram.json, docs/scheme.png
bcb5740	02.11.2026	добавлен документ в firmware	firmware/variant_06.ino
decb0eb	02.11.2026	изменена schema.sql	db/schema.sql, ER-диаграмма
f65aa69	02.11.2026	добавлен queries.sql	db/queries.sql
474ca67	02.11.2026	Добавлен поток Node-RED и журнал данных	nodered/flow.json, data/log.txt
9c722cc	02.11.2026	Добавлена проверка неисправности и документация	data/log_fault.txt, инструкция пользователя
7bf4cd2	04.11.2026	изменен .gitignore	Исключение db/variant.db и load.sql из репозитория

Выводы

В ходе курсового проекта разработана и проверена виртуальная система сбора и учёта данных для сценария «Склад зерна» (вариант №6). Получены следующие результаты:

- выполнена структурная схема и алгоритм работы, определён текстовый протокол обмена с контроллером;
- подобраны компоненты, рассчитан резистор R1= 150 Ом для красного светодиода;
- принципиальная схема выполнена в EasyEDA по требованиям ЕСКД, модель собрана в Wokwi;
- написана программа для Arduino Uno, которая опрашивает три датчика каждые 2 с и формирует события при влажности выше 70 % только в момент смены состояния;
- спроектирована база данных SQLite из двух таблиц и подготовлены три контрольных запроса;
- настроен поток Node-RED, автоматически и повторяемо загружающий журнал в базу и фиксирующий неисправности датчиков;
- тестирование на штатном и аварийном сценариях подтвердило правильность работы;
- оценён объём данных – около 5,2 МБ в сутки, проект оформлен в репозитории Git.

Разработанный проект может быть использован как основа для других сценариев: при смене датчиков достаточно изменить константы и функции чтения в программе, диапазоны и период в функции Node-RED, а структура базы данных остаётся прежней.

Список используемых сокращений

Сокращения	Расшифровка
АЦП	аналого-цифровой преобразователь
БД	база данных
ГОСТ	государственный стандарт
ЕСКД	единая система конструкторской документации
ИИС	интеллектуальные интегрированные системы
СУБД	система управления базами данных
NTC	термистор с отрицательным температурным коэффициентом
PIR	пассивный инфракрасный датчик движения
SQL	язык структурированных запросов
UART	универсальный асинхронный приёмопередатчик

Список используемой литературы

1) Блум, А. Изучаем Arduino: инструменты и методы технического волшебства / А. Блум ; пер. с англ. – Санкт-Петербург : БХВ-Петербург, 2015. – 336 с. – ISBN 978-5-9775-3422-9. – Текст : непосредственный.

2) Петин, В. А. Интернет вещей на базе Arduino и Raspberry Pi / В. А. Петин. – Санкт-Петербург : БХВ-Петербург, 2018. – 256 с. – ISBN 978-5-9775-6333-5. – Текст : непосредственный.

3) Arduino Language Reference. – Текст : электронный // Arduino. – URL: https://www.arduino.cc/reference/ (дата обращения: 28.10.2026).

4) EasyEDA Documentation. – Текст : электронный // EasyEDA. – URL: https://docs.easyeda.com/en/ (дата обращения: 28.10.2026).

5) Node-RED Documentation. – Текст : электронный // Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 28.10.2026).

6) SQLite Documentation. – Текст : электронный // SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 28.10.2026).

7) Wokwi Docs. – Текст : электронный // Wokwi. – URL: https://docs.wokwi.com (дата обращения: 28.10.2026).

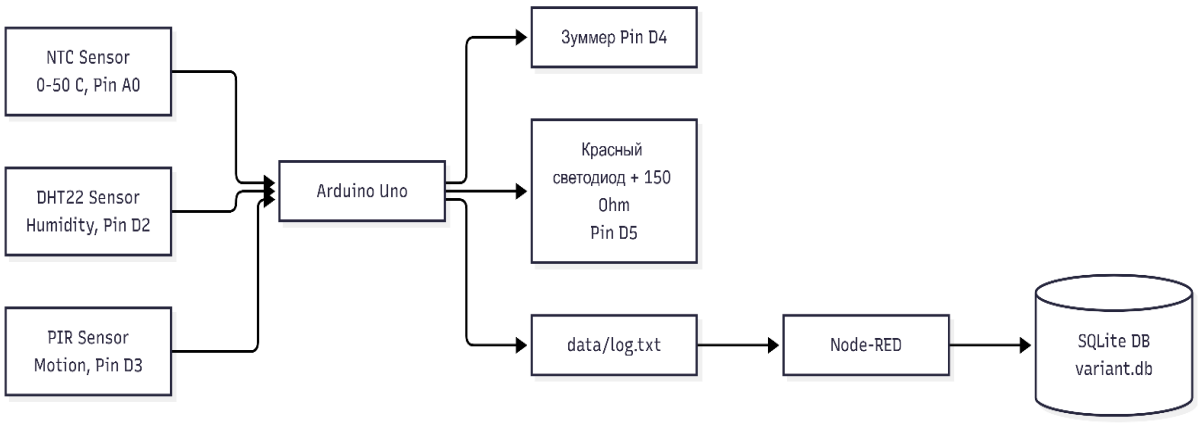
8) Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – Текст : электронный // Git SCM. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 28.10.2026).

9) Репозиторий проекта «Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino (Вариант №)». – Текст : электронный // GitHub. – URL: https://github.com/Namiellee/Gubatova1 (дата обращения: 02.11.2026).

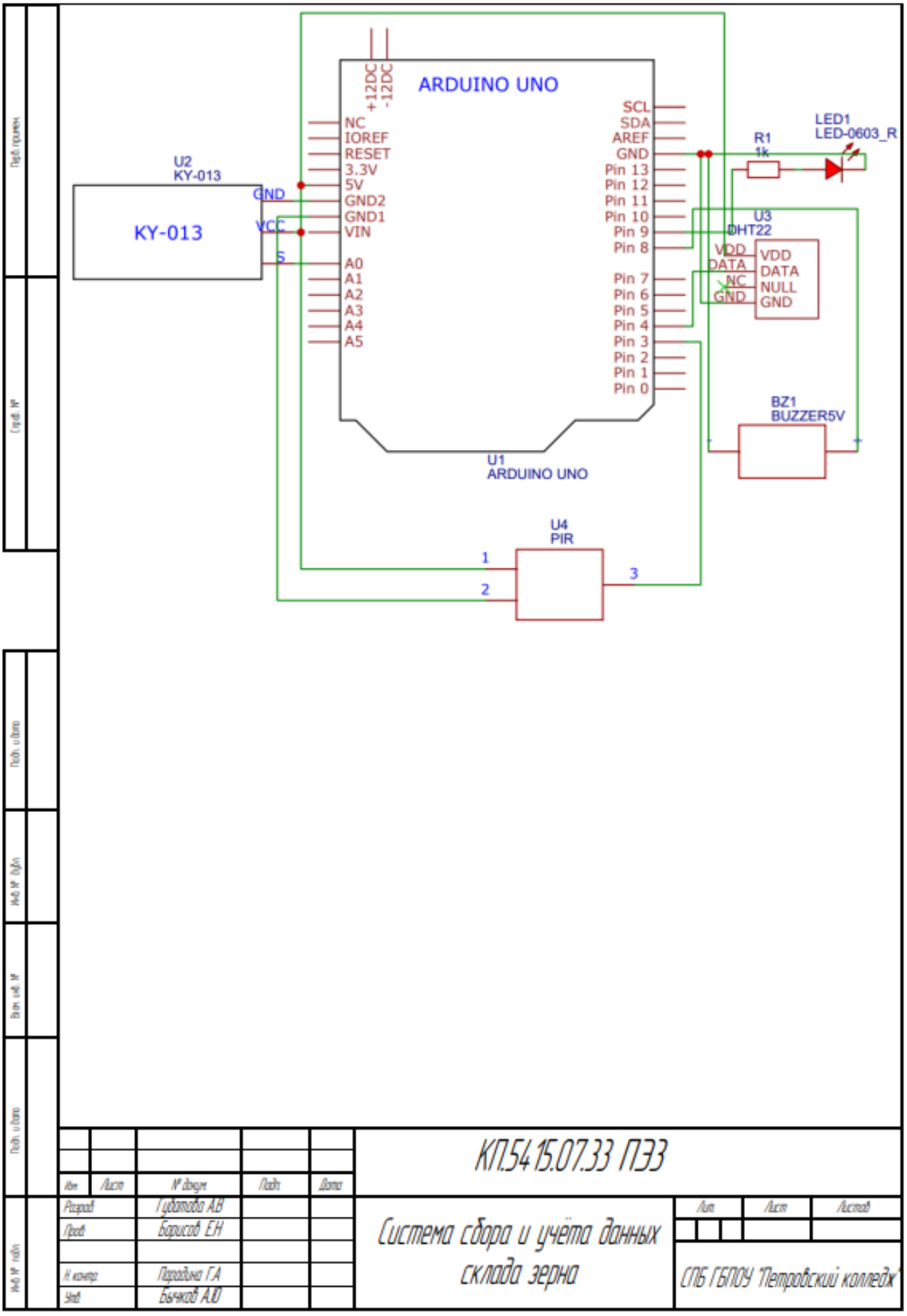
10) Модель виртуальной системы сбора и учёта данных в симуляторе Wokwi. – Текст : электронный // Wokwi. – URL: https://wokwi.com/projects/479756790403518465 (дата обращения: 02.11.2026).

ПРИЛОЖЕНИЕ А

Структурная схема



ПРИЛОЖЕНИЕ Б



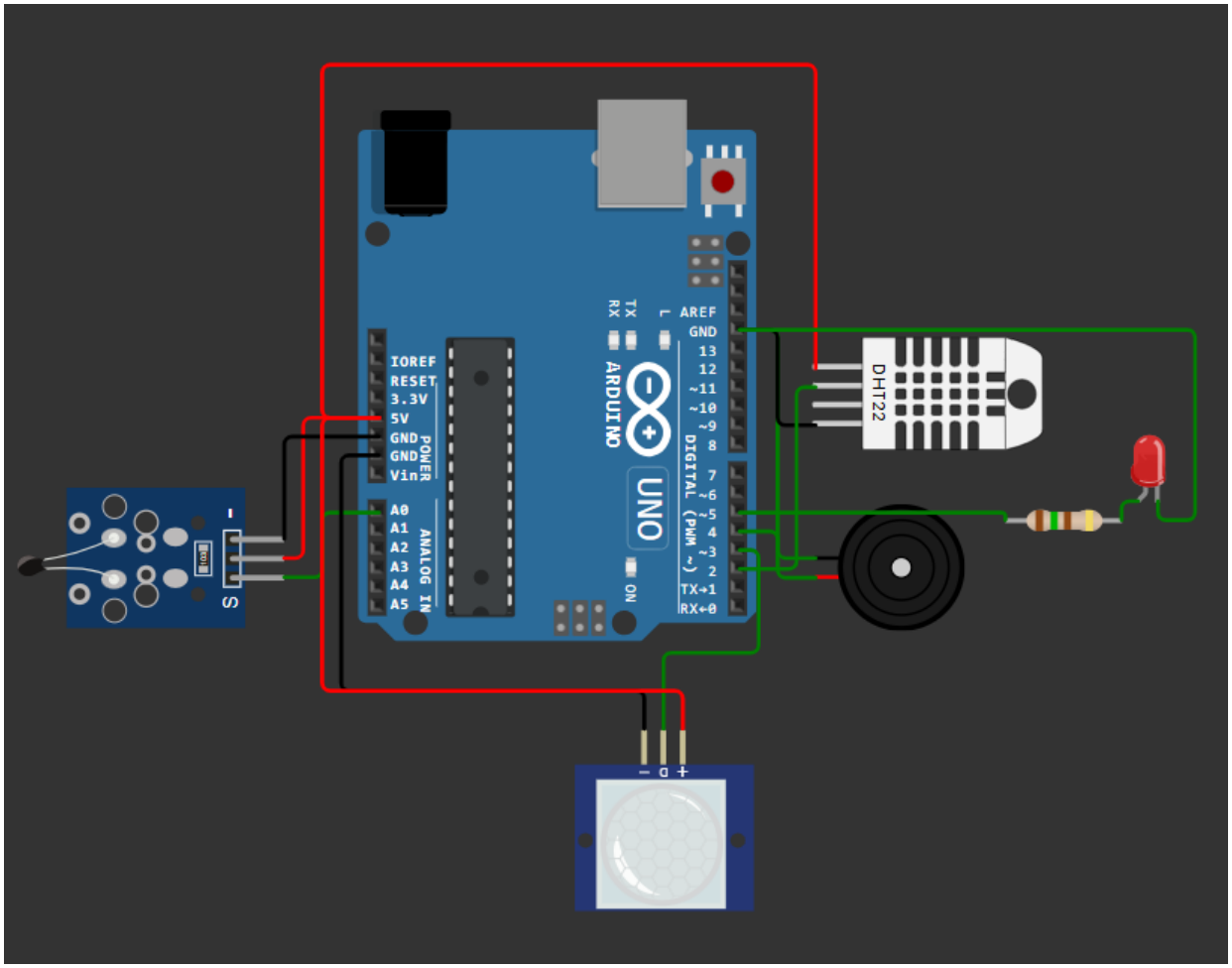
ПРИЛОЖЕНИЕ В

Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Приме-чание
				Светодиоды		
			LED1	Светодиод LED-TH-3mm красный	1	
				Модули и платы		
			U1	ARDUINO UNO	1	
			U2	NTC термистор 10 кОм	1	
			U3	DHT22 датчик температуры и влажности	1	
			U4	PIR датчик движения HC-SR501	1	
			BZ1	Зуммер пассивный 5 В	1	
				Резисторы		
			R1	Резистор 150Ом	1	

Подп. и дата	Инв. № дубл	Взаим. инв. №	Подп. и дата	Инв. № подл.

Изм	Лист	№ докум.	Подп.	Дата	КП.5415.07.33 ПЗЗ		
Разраб.		Губатова А.В			Система сбора и учёта данных склада	Лит.	Лист 1 / Листов 1
Пров.		Борисов Е.Н				СПБ ГБПОУ «Петровский колледж»	
Н.контр.		Парадина Г.А					
Утв.		Бычков А.Ю					

Приложение Г Модель в Wokwi



Приложение Д Листинг программы Arduino (variant_6.ino)

```
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT22
#define PIR_PIN 3
#define BUZZER_PIN 4
#define LED_PIN 5
#define NTC_PIN A0
#define T_MS 2000
#define THRESHOLD_HUMIDITY 70.0
DHT dht(DHTPIN, DHTTYPE);
int n = 0;
bool thresholdActive = false;
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
}
double readNTC() {
  int raw = analogRead(NTC_PIN);
  return 0.0 + (50.0 - 0.0) * raw / 1023.0;
}
void loop() {
  n++;
  double s1 = readNTC();
  double s2 = dht.readHumidity();
```

```
  int s3 = digitalRead(PIR_PIN);
  if (isnan(s1)) s1 = 0.0;
  if (isnan(s2)) s2 = 0.0;
  Serial.print("DATA n=");
  Serial.print(n);
  Serial.print(" s1=");
  Serial.print(s1, 2);
  Serial.print(" s2=");
  Serial.print(s2, 2);
  Serial.print(" s3=");
  Serial.println(s3);
  if (s2 > THRESHOLD_HUMIDITY) {
    if (!thresholdActive) {
      digitalWrite(BUZZER_PIN, HIGH);
      digitalWrite(LED_PIN, HIGH);
      Serial.print("EVENT n=");
      Serial.print(n);
      Serial.println(" type=THRESHOLD");
      thresholdActive = true;}
  } else {
    if (thresholdActive) {
      digitalWrite(BUZZER_PIN, LOW);
      digitalWrite(LED_PIN, LOW);
      Serial.print("EVENT n=");
      Serial.print(n);
      Serial.println(" type=NORMAL");
      thresholdActive = false;}
    }
  delay(T_MS);
}
```

Приложение Е Листинг функции разбора журнала Node-RED

```
const T = 2;
const RANGES = {
   s1: [0, 50],
   s2: [0, 100],
   s3: [0, 1] };
const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];
text.split(/\r?\n/).forEach(raw => {
   const line = raw.trim();
   if (!line) return;
   const parts = line.split(' ');
   const kind = parts[0];
   const f = {};
   parts.slice(1).forEach(p => {
      const kv = p.split('=');
      if (kv.length === 2) {
         f[kv[0]] = kv[1]; }
   });
   const n = parseInt(f.n);
   if (isNaN(n)) return;
   const t_s = n * T;
   if (kind === 'DATA') {
      ['s1', 's2', 's3'].forEach(s => {
         const v = parseFloat(f[s]);
         if (!isNaN(v)) {
                        sql.push(`INSERT INTO readings(n,t_s,sensor,value)
VALUES(${n},${t_s},'${s}',${v});`);
         }
         const [min, max] = RANGES[s];
```

```
            if (isNaN(v) || v < min || v > max) {
                const safeMsg = `датчик ${s} вне диапазона [${min};${max}]`.replace(/'/g, "''");
                sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t_s},'FAULT','${safeMsg}');`);
            }
        });
    }
    else if (kind === 'EVENT') {
        const type = f.type || 'UNKNOWN';
        let message = '';
        if (type === 'THRESHOLD') {
            message = 'влажность > 70%';
        } else if (type === 'NORMAL') {
            message = 'влажность в норме';
        } else {
            message = 'событие контроллера';
        }
        const safeMsg = message.replace(/'/g, "''");
            sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t_s},'${type}','${safeMsg}');`);
    }
});
const head =
    'BEGIN;\n' +
    'DELETE FROM readings;\n' +
    'DELETE FROM events;\n' +
    "DELETE FROM sqlite_sequence WHERE name IN ('readings','events');\n";
return [null, { payload: head + sql.join('\n') + '\nCOMMIT;\n' }];
return msg;
```

Приложение Ж Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings(
   id      INTEGER PRIMARY KEY AUTOINCREMENT,
   n       INTEGER NOT NULL,
   t_s     REAL    NOT NULL,
   sensor  TEXT    NOT NULL,
   value   REAL    NOT NULL
);
CREATE TABLE IF NOT EXISTS events(
   id      INTEGER PRIMARY KEY AUTOINCREMENT,
   n       INTEGER NOT NULL,
   t_s     REAL    NOT NULL,
   type    TEXT    NOT NULL,
   message TEXT
);
CREATE INDEX IF NOT EXISTS idx_readings_n ON readings(n);
CREATE INDEX IF NOT EXISTS idx_events_n   ON events(n);
```

db/queries/sql

```
SELECT n, t_s, sensor, value
FROM readings
ORDER BY n DESC, sensor
LIMIT 10;
SELECT sensor,
      MIN(value)   AS min_val,
      AVG(value)   AS avg_val,
      MAX(value)   AS max_val
FROM readings
GROUP BY sensor;
SELECT n, t_s, type, message
FROM events
ORDER BY n, id;
```

Приложение И Журнал Serial и загрузочный скрипт data/log.txt

DATA n=1 s1=25.56 s2=40.00 s3=0

DATA n=2 s1=25.56 s2=40.00 s3=0

DATA n=3 s1=25.56 s2=40.00 s3=0

DATA n=4 s1=25.56 s2=40.00 s3=0

DATA n=5 s1=25.56 s2=40.00 s3=0

DATA n=6 s1=25.56 s2=40.00 s3=0

DATA n=7 s1=25.56 s2=40.00 s3=0

DATA n=8 s1=25.56 s2=40.00 s3=0

DATA n=9 s1=25.56 s2=40.00 s3=0

DATA n=10 s1=25.56 s2=40.00 s3=0

DATA n=11 s1=25.56 s2=40.00 s3=0

DATA n=12 s1=25.56 s2=40.00 s3=0

DATA n=13 s1=25.56 s2=75.0 s3=0

EVENT n=13 type=THRESHOLD

DATA n=14 s1=25.56 s2=78.2 s3=0

DATA n=15 s1=25.56 s2=68.0 s3=0

EVENT n=15 type=NORMAL

Тест неисправности: строка журнала

DATA n=16 s1=999.0 s2=40.0 s3=0

db/load.sql

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,2,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(2,4,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(2,4,'s2',40);
```

```
INSERT INTO readings(n,t_s,sensor,value) VALUES(2,4,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,10,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,10,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,10,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,12,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,12,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,12,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,14,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,14,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,14,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,16,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,16,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,16,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,18,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,18,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,18,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,22,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,22,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,22,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,24,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,24,'s2',40);
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,24,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,26,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,26,'s2',75);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,26,'s3',0);
INSERT INTO events(n,t_s,type,message) VALUES(13,26,'THRESHOLD','влажность > 70%');
INSERT INTO readings(n,t_s,sensor,value) VALUES(14,28,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(14,28,'s2',78.2);
INSERT INTO readings(n,t_s,sensor,value) VALUES(14,28,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(15,30,'s1',25.56);
INSERT INTO readings(n,t_s,sensor,value) VALUES(15,30,'s2',68);
INSERT INTO readings(n,t_s,sensor,value) VALUES(15,30,'s3',0);
INSERT INTO events(n,t_s,type,message) VALUES(15,30,'NORMAL','влажность в норме');
COMMIT;
```

Приложение К

История Git

Хеш	Дата	Сообщение коммита	Этап / Примечание
b9a25a6	29.10.2026	Initial commit	Создание репозитория, структура папок, README.md
c076db0	29.10.2026	добавлена структурная схема	docs/structure.png (Этап 1)
6614e2f	02.11.2026	Добавлена схема в Wokwi	sim/diagram.json, docs/scheme.png (Этап 2)
bcb5740	02.11.2026	добавлен документ в firmware	firmware/variant_06.ino (Этап 3)
decb0eb	02.11.2026	изменена schema.sql	db/schema.sql, ER-диаграмма (Этап 5)
f65aa69	02.11.2026	добавлен queries.sql	db/queries.sql (Этап 5)
474ca67	02.11.2026	Добавлен поток Node-RED и журнал данных	nodered/flow.json, data/log.txt (Этап 6)
9c722cc	02.11.2026	Добавлена проверка неисправности и документация	data/log_fault.txt, инструкция пользователя (Этапы 7–8)
7bf4cd2	04.11.2026	изменен .gitignore	Исключение db/variant.db и load.sql из репозитория